یادگیری و تمرین برای مصاحبه‌های برنامه‌نویسی

گردآورنده: علی عالی پور

1404-03-20

Solid

الگوی SOLID مجموعه‌ای از پنج اصل طراحی در برنامه‌نویسی شیءگرا است که برای ایجاد کدهای تمیز، قابل نگهداری و مقیاس‌پذیر طراحی شده‌اند.

1.Single Responsibility Principle

هر کلاس یا ماژول باید فقط یک دلیل برای تغییر داشته باشه. به عبارت دیگه، هر کلاس باید فقط یک کار یا مسئولیت خاص رو انجام بده.

مثال: یه کلاس که هم داده رو از دیتابیس می‌خونه و هم گزارش چاپ می‌کنه، این اصل رو نقض می‌کنه. بهتره این مسئولیت‌ها به دو کلاس جدا تقسیم بشن.**مثال نقص با SRP**

csharp Collapse Wrap Copy

```csharp
public class UserManager
{
    public void CreateUser(string name, string email)
    {
        // منطق ایجاد کاربر
        Console.WriteLine($"User {name} created with email {email}");

        // ذخیره در دیتابیس
        SaveToDatabase(name, email);

        // ارسال ایمیل
        SendWelcomeEmail(email);
    }

    private void SaveToDatabase(string name, string email)
    {
        Console.WriteLine($"Saving {name} to database...");
        // کد ذخیره‌سازی
    }

    private void SendWelcomeEmail(string email)
    {
        Console.WriteLine($"Sending welcome email to {email}...");
        // کد ارسال ایمیل
    }
}
```

انجام با پیاده سازی SRP

csharp Collapse Wrap Copy

```csharp
// کلاس مدیریت کاربر (فقط منطق تجاری)
public class UserManager
{
    private readonly IUserRepository _userRepository;
    private readonly IEmailService _emailService;

    public UserManager(IUserRepository userRepository, IEmailService emailService)
    {
        _userRepository = userRepository;
        _emailService = emailService;
    }

    public void CreateUser(string name, string email)
    {
        // منطق ایجاد کاربر
        var user = new User { Name = name, Email = email };
        Console.WriteLine($"User {name} created with email {email}");

        // واگذاری ذخیره‌سازی به Repository
        _userRepository.Save(user);

        // واگذاری ارسال ایمیل به سرویس ایمیل
        _emailService.SendWelcomeEmail(email);
    }
}
```

2. Open/Closed Principle

کلاس‌ها باید برای گسترش باز (قابل افزودن قابلیت جدید) و برای تغییر بسته (بدون نیاز به تغییر کد موجود) باشن.

مثال: فرض کنید سیستمی برای محاسبه مساحت اشکال هندسی داریم.

```
public interface IShape
{
    double CalculateArea();
}

public class Rectangle : IShape
{
    public double Width { get; set; }
    public double Height { get; set; }

    public double CalculateArea()
    {
        return Width * Height;
    }
}

public class Circle : IShape
{
    public double Radius { get; set; }

    public double CalculateArea()
    {
        return Math.PI * Radius * Radius;
    }
}

public class Triangle : IShape
{
    public double Base { get; set; }
    public double Height { get; set; }

    public double CalculateArea()
    {
        return 0.5 * Base * Height;
    }
}
```

```
public class ShapeCalculator
{
    public double CalculateArea(IShape shape)
    {
        return shape.CalculateArea(); // بدون نیاز به تغییر کد، هر شکل مساحت خودش را محاسبه می‌کند
    }
}

class Program
{
    static void Main(string[] args)
    {
        ShapeCalculator calculator = new ShapeCalculator();

        IShape rectangle = new Rectangle { Width = 5, Height = 3 };
        Console.WriteLine($"Rectangle Area: {calculator.CalculateArea(rectangle)}"); // خروجی: Rectangle Area: 15

        IShape circle = new Circle { Radius = 2 };
        Console.WriteLine($"Circle Area: {calculator.CalculateArea(circle)}"); // خروجی: Circle Area: 12.566...

        IShape triangle = new Triangle { Base = 4, Height = 6 };
        Console.WriteLine($"Triangle Area: {calculator.CalculateArea(triangle)}"); // خروجی: Triangle Area: 12
    }
}
```

3. Liskov Substitution Principle

اشیاء یک کلاس فرزند باید بتونن بدون مشکل جایگزین اشیاء کلاس والد بشن، بدون اینکه رفتار برنامه تغییر کنه.

مثال: اگه یه کلاس Bird داری که متد fly داره، کلاس فرزند Penguin نباید این متد رو به‌گونه‌ای پیاده‌سازی کنه که خطا بده (چون پنگوئن پرواز نمی‌کنه). بهتره طراحی رو اصلاح کنی (مثلاً متد fly رو فقط برای پرنده‌های پروازی تعریف کنی).

```
public interface IFlyable
{
    void Fly();
}

public abstract class Bird
{
    public virtual void MakeSound()
    {
        Console.WriteLine("Making bird sound...");
    }
}

public class Sparrow : Bird, IFlyable
{
    public void Fly()
    {
        Console.WriteLine("Sparrow is flying!");
    }

    public override void MakeSound()
    {
        Console.WriteLine("Chirp chirp!");
    }
}
```

```csharp
public class BirdWatcher
{
    public void MakeBirdFly(IFlyable bird)
    {
        bird.Fly(); // فقط برای پرندگانی که می‌توانند پرواز کنند
    }

    public void MakeBirdSound(Bird bird)
    {
        bird.MakeSound(); // برای همه پرندگان
    }
}

class Program
{
    static void Main(string[] args)
    {
        BirdWatcher watcher = new BirdWatcher();

        Sparrow sparrow = new Sparrow();
        watcher.MakeBirdFly(sparrow); // خروجی: Sparrow is flying!
        watcher.MakeBirdSound(sparrow); // خروجی: Chirp chirp!

        Ostrich ostrich = new Ostrich();
        watcher.MakeBirdSound(ostrich); // خروجی: Boom boom!
        // نمی‌توانیم MakeBirdFly را برای Ostrich چون، فراخوانی کنیم IFlyable نیست
    }
}
```

4. Interface Segregation Principle

کلاس‌ها نباید مجبور بشن رابط‌هایی (Interfaces) رو پیاده‌سازی کنن که بهشون نیاز ندارن.

فرض کنید می‌خواهیم سیستمی برای مدیریت پرینترها طراحی کنیم. ابتدا یک رابط بزرگ و غیراصولی طراحی می‌کنیم:

csharp Collapse Wrap Copy

```csharp
public interface IPrinter
{
    void Print();
    void Scan();
    void Fax();
}
```

حالا فرض کنید دو نوع پرینتر داریم:

1- پرینتر ساده که فقط چاپ می‌کند
2- پرینتر چندکاره که چاپ، اسکن و فکس می‌کند.

اگر کلاسی برای پرینتر ساده بنویسیم، مجبور می‌شویم متدهای Scan و Fax را هم پیاده‌سازی کنیم، حتی اگر پرینتر ساده این قابلیت‌ها را نداشته باشد:

csharp Collapse Wrap Copy

```csharp
public class SimplePrinter : IPrinter
{
    public void Print()
    {
        Console.WriteLine("Printing document...");
    }

    public void Scan()
    {
        throw new NotImplementedException(); // این پرینتر اسکن نمی‌کند!
    }

    public void Fax()
    {
        throw new NotImplementedException(); // این پرینتر فکس نمی‌کند!
    }
}
```

اینجا می‌بینیم که پرینتر ساده مجبور به پیاده‌سازی متدهایی شده که اصلاً به آن‌ها نیاز ندارد. این نقض اصل ISP است. برای رعایت ISP، رابط را به بخش‌های کوچک‌تر و تخصصی تقسیم می‌کنیم

csharp Collapse Wrap Copy

```csharp
public interface IPrint
{
    void Print();
}

public interface IScan
{
    void Scan();
}

public interface IFax
{
    void Fax();
}
```

حالا هر کلاس می‌تواند فقط رابط‌هایی را پیاده‌سازی کند که به آن نیاز دارد. برای مثال:

csharp Collapse Wrap Copy

```csharp
public class SimplePrinter : IPrint
{
    public void Print()
    {
        Console.WriteLine("Printing document...");
    }
}

public class MultiFunctionPrinter : IPrint, IScan, IFax
{
    public void Print()
    {
        Console.WriteLine("Printing document...");
    }

    public void Scan()
    {
        Console.WriteLine("Scanning document...");
    }

    public void Fax()
    {
        Console.WriteLine("Sending fax...");
    }
}
```

5. Dependency Inversion Principle

ماژول‌های سطح بالا نباید به ماژول‌های سطح پایین وابسته باشند؛ هر دو باید به abstractions (رابط‌ها یا کلاس‌های abstract) وابسته باشند. همچنین، abstractions نباید به جزئیات وابسته باشند، بلکه جزئیات باید به abstractions وابسته باشند.

فرض کنید سیستمی برای ارسال اعلان‌ها (Notifications) داریم. ابتدا یک پیاده‌سازی غیراصولی و بدون رعایت DIP می‌نویسیم:

```csharp
public class EmailService
{
    public void SendEmail(string message)
    {
        Console.WriteLine($"Sending email: {message}");
    }
}

public class NotificationManager
{
    private readonly EmailService _emailService;

    public NotificationManager()
    {
        _emailService = new EmailService(); // وابستگی مستقیم به EmailService
    }

    public void SendNotification(string message)
    {
        _emailService.SendEmail(message);
    }
}
```

ر این کد، NotificationManager به صورت مستقیم به کلاس EmailService وابسته است. اگر بخواهیم نوع اعلان را تغییر دهیم (مثلاً به جای ایمیل، پیامک یا نوتیفیکیشن Push بفرستیم)، باید کد NotificationManager را تغییر دهیم. این نقض اصل DIP است.

راه‌حل با رعایتDIP

برای رعایتDIP ، یک رابط (abstraction) تعریف می‌کنیم و وابستگی‌ها را به آن منتقل می‌کنیم.

csharp Collapse Wrap Copy

```csharp
public interface INotificationService
{
    void Send(string message);
}

public class EmailService : INotificationService
{
    public void Send(string message)
    {
        Console.WriteLine($"Sending email: {message}");
    }
}

public class SmsService : INotificationService
{
    public void Send(string message)
    {
        Console.WriteLine($"Sending SMS: {message}");
    }
}
```

```csharp
public class NotificationManager
{
    private readonly INotificationService _notificationService;

    public NotificationManager(INotificationService notificationService)
    {
        _notificationService = notificationService; // تزریق وابستگی
    }

    public void SendNotification(string message)
    {
        _notificationService.Send(message);
    }
}
```

OOP

برنامه‌نویسی شیءگرا یه روش برنامه‌نویسیه که داده‌ها و رفتارها رو در قالب اشیا (Objects) سازمان‌دهی می‌کنه. این اشیا از کلاس‌ها ساخته می‌شن.

اصول اصلی OOP

کپسوله‌سازی (Encapsulation):

داده‌ها و متدهای مرتبط رو توی یه کلاس جمع می‌کنه و دسترسی به داده‌ها رو از طریق متدها کنترل می‌کنه (مثل استفاده از private و public).

چرا مهمه؟ محافظت از داده‌ها و پنهان کردن جزئیات داخلی.

ارث‌بری (Inheritance):

یه کلاس می‌تونه ویژگی‌ها و رفتارهای یه کلاس دیگه رو به ارث ببره.

چرا مهمه؟ کاهش تکرار کد و ایجاد سلسله‌مراتب منطقی.

چندریختی (Polymorphism)

امکان استفاده از یه رابط یا متد به شکل‌های مختلف (مثل override کردن متدها یا استفاده از رابط‌ها)

انتزاع (Abstraction)

فقط جزئیات ضروری رو نشون می‌ده و پیچیدگی‌های غیرضروری رو مخفی می‌کنه.

مثال با C#

```
// اصل انتزاع: تعریف یه رابط برای رفتارهای مشترک حیوانات
public interface IAnimal
{
    string MakeSound(); // هر حیوان باید بتونه صدا بده
    string GetName();   // هر حیوان یه اسم داره
}

// کلاس پایه برای حیوانات (اصل ارث‌بری)
public abstract class Animal : IAnimal
{
    // اصل کپسوله‌سازی: فیلد خصوصی با پراپرتی عمومی
    private string name;
    public Animal(string name)
    {
        this.name = name;
```

```
    }
    public string GetName()
    {
        return name;
    }
    // متد انتزاعی برای صدا (باید در کلاس‌های فرزند پیاده‌سازی بشه)
    public abstract string MakeSound();
}

// کلاس شیر (با چندریختی اصل override)
public class Lion : Animal
{
    public Lion(string name) : base(name) { }
    public override string MakeSound()
    {
        return "Roar!";
    }
}

// کلاس پرنده (با چندریختی اصل override)
public class Bird : Animal
{
    public Bird(string name) : base(name) { }

    public override string MakeSound()
    {
        return "Chirp!";
    }
}
// برنامه اصلی
class Program
{
    static void Main(string[] args)
    {
        // ساختن اشیا از کلاس‌های مختلف
        IAnimal lion = new Lion("Simba");
        IAnimal bird = new Bird("Tweety");

        // استفاده از چندریختی: هر حیوان صدای خاص خودش رو تولید می‌کنه
        Console.WriteLine($"{lion.GetName()} says: {lion.MakeSound()}");
        Console.WriteLine($"{bird.GetName()} says: {bird.MakeSound()}");
    }
}
```

TDD

TDD یه روش توسعه نرم‌افزاره که توش اول تست‌ها رو می‌نویسی و بعد کدی می‌نویسی که اون تست‌ها رو پاس کنه. این روش روی نوشتن کد تمیز، قابل اطمینان و قابل نگهداری تمرکز داره. چرخه TDD معمولاً به این شکله معروف به Red-Green-Refactor

Red (تست شکست بخوره): یه تست می‌نویسی که عملکرد مورد انتظارت رو بررسی کنه. چون هنوز کدی برای اون عملکرد ننوشتی، تست شکست می‌خوره.

Green (تست پاس بشه): کمترین مقدار کد رو می‌نویسی تا تست پاس بشه.

Refactor (بازسازی): کد رو بهینه و تمیز می‌کنی (بدون تغییر رفتار) و مطمئن می‌شی که تست‌ها همچنان پاس می‌شن.

نوشتن اولین تست (Red Phase)

اول یه تست برای متد جمع می‌نویسیم. چون هنوز کلاس Calculator وجود نداره، این تست نمی‌تونه کامپایل بشه.

```
using Xunit;
public class CalculatorTests
{
    [Fact]
    public void Add_WhenGivenTwoNumbers_ReturnsTheirSum()
    {
        // Arrange
        var calculator = new Calculator();
        int a = 3;
        int b = 5;
        // Act
        int result = calculator.Add(a, b);
        // Assert
        Assert.Equal(8, result);
    }
}
```

قدم ۲: پیاده‌سازی حداقل کد (Green Phase)

حالا کلاس Calculator رو با متد Add می‌سازیم تا تست پاس بشه.

```
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }
}
```

قدم ۳: اضافه کردن تست جدید (Red Phase)

```
using Xunit;
public class CalculatorTests
{
    [Fact]
    public void Add_WhenGivenTwoNumbers_ReturnsTheirSum()
    {
        var calculator = new Calculator();
        int a = 3;
        int b = 5;
        int result = calculator.Add(a, b);
        Assert.Equal(8, result);
    }



    [Fact]
    public void Multiply_WhenGivenTwoNumbers_ReturnsTheirProduct()
    {
        var calculator = new Calculator();
        int a = 4;
        int b = 3;
        int result = calculator.Multiply(a, b);
        Assert.Equal(12, result);
    }
}
```

Reflection

گاهی اوقات نیاز است که در زمان اجرا، بدون نیاز به دسترسی مستقیم به کد، عملیات‌هایی مانند مشاهده یا تغییر متدها و متغیرها انجام شود. Reflection در سی شارپ این امکان را فراهم می‌کند که کدهایی بنویسید که بتوانند ساختار برنامه را در زمان اجرا بررسی کرده و براساس آن، تصمیمات مختلفی بگیرند.

نحوه استفاده از Reflection

مثال 1: دسترسی به اطلاعات کلاس

در این مثال، ما از Reflection برای بدست آوردن نام یک کلاس استفاده می‌کنیم.

```
public class Person

{

  public string Name { get; set; }

  public int Age { get; set; }

}

var type = typeof(Person);

Console.WriteLine("نام کلاس: " + type.Name);
```

مثال 2: مشاهده تمام متدهای یک کلاس

در این مثال، با استفاده ازReflection ، تمام متدهای کلاس Person را در زمان اجرا مشاهده می‌کنیم.

```
var methods = typeof(Person).GetMethods();

foreach (var method in methods)

{

  Console.WriteLine("نام متد: " + method.Name);

}
```

Design Pattern

دیزاین پترن‌ها (Design Patterns) در برنامه‌نویسی، راه‌حل‌های استاندارد و آزمایش‌شده‌ای برای مشکلات رایج طراحی نرم‌افزار هستند. این الگوها به توسعه‌دهندگان کمک می‌کنند تا کدهایی تمیز، قابل‌نگهداری، و انعطاف‌پذیر بنویسند. دیزاین پترن‌ها دستورالعمل‌های کلی هستند که به زبان یا پلتفرم خاصی وابسته نیستند.

تعریف دیزاین پترن

دیزاین پترن‌ها راه‌حل‌های تکرارشونده برای مسائل متداول در طراحی نرم‌افزار هستند که در کتاب معروف " Design Patterns: Elements of Reusable Object-Oriented Software" نوشته‌ی گروه چهار نفره (Gang of Four یا GoF) معرفی شدند. این کتاب 23 الگوی اصلی را در سه دسته‌بندی کلی ارائه می‌دهد.

Creational Patterns

این الگوها با نحوه‌ی ایجاد اشیاء سروکار دارند و انعطاف‌پذیری و استفاده مجدد را در فرآیند ایجاد بهبود می‌بخشند.

Singleton: تضمین می‌کند که از یک کلاس فقط یک نمونه (Instance) وجود داشته باشد.

Factory Method: ایجاد اشیاء را به زیرکلاس‌ها واگذار می‌کند.

Abstract Factory: مجموعه‌ای از اشیاء مرتبط را بدون مشخص کردن کلاس‌هایشان ایجاد می‌کند.

Builder: فرآیند ساخت اشیاء پیچیده را مرحله‌به‌مرحله انجام می‌دهد.

Prototype: اشیاء جدید را با کپی کردن یک نمونه موجود می‌سازد.

Structual Patterns

این الگوها به چگونگی ترکیب کلاس‌ها و اشیاء برای تشکیل ساختارهای بزرگ‌تر می‌پردازند.

Adapter: امکان همکاری بین دو رابط ناسازگار را فراهم می‌کند.

Decorator: قابلیت‌های جدید را به‌صورت پویا به یک شیء اضافه می‌کند.

Façade: رابط ساده‌ای برای زیرسیستم‌های پیچیده فراهم می‌کند.

Composite: اشیاء را به‌صورت ساختار درختی سازمان‌دهی می‌کند.

Proxy: یک شیء واسطه برای کنترل دسترسی به شیء اصلی ایجاد می‌کند.

Behavioral Patterns

این الگوها به تعاملات و مسئولیت‌های بین اشیاء تمرکز دارند.

Observer: وقتی حالت یک شیء تغییر می‌کند، اشیاء وابسته به آن به‌روز می‌شوند

Strategy: امکان انتخاب الگوریتم‌های مختلف در زمان اجرا را فراهم می‌کند.

Command: درخواست‌ها را به‌صورت اشیاء کپسوله می‌کند.

Template Method: ساختار کلی یک الگوریتم را تعریف می‌کند و جزئیات را به زیرکلاس‌ها واگذار می‌کند.

Iterator: دسترسی به عناصر یک مجموعه را بدون نمایش ساختار داخلی آن فراهم می‌کند

1. Singleton Pattern

```csharp
public class Singleton
{
    private static Singleton _instance;
    private static readonly object _lock = new object();

    private Singleton() { } // سازنده خصوصی

    public static Singleton Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance == null)
                {
                    _instance = new Singleton();
                }
                return _instance;
            }
        }
    }

    public void DoSomething() => Console.WriteLine("Singleton is working!");
}

class Program
{
    static void Main()
    {
        Singleton s1 = Singleton.Instance;
        Singleton s2 = Singleton.Instance;
        Console.WriteLine(s1 == s2); // خروجی: True
        s1.DoSomething();
    }
}
```

2.Factory Method Pattern

```csharp
public abstract class Vehicle
{
    public abstract void Drive();
}

public class Car : Vehicle
{
    public override void Drive() => Console.WriteLine("Driving a Car");
}

public class Bike : Vehicle
{
    public override void Drive() => Console.WriteLine("Driving a Bike");
}

public abstract class VehicleFactory
{
    public abstract Vehicle CreateVehicle();
}

public class CarFactory : VehicleFactory
{
    public override Vehicle CreateVehicle() => new Car();
}

public class BikeFactory : VehicleFactory
{
    public override Vehicle CreateVehicle() => new Bike();
}

class Program
{
    static void Main()
    {
        VehicleFactory factory = new CarFactory();
        Vehicle vehicle = factory.CreateVehicle();
        vehicle.Drive(); // خروجی: Driving a Car
    }
}
```

3. Observer Pattern

این الگو برای اطلاع‌رسانی تغییرات یک شیء به اشیاء وابسته استفاده می‌شود.

```csharp
using System;
using System.Collections.Generic;

public class Subject
{
    private readonly List<IObserver> _observers = new List<IObserver>();
    private int _state;

    public int State
    {
        get => _state;
        set
        {
            _state = value;
            Notify();
        }
    }

    public void Attach(IObserver observer) => _observers.Add(observer);
    public void Notify()
    {
        foreach (var observer in _observers)
        {
            observer.Update();
        }
    }
}
```

```
public interface IObserver
{
    void Update();
}

public class ConcreteObserver : IObserver
{
    private readonly Subject _subject;
    public ConcreteObserver(Subject subject) => _subject = subject;
    public void Update() => Console.WriteLine($"Subject's state changed to: {_subject.State}");
}

class Program
{
    static void Main()
    {
        Subject subject = new Subject();
        ConcreteObserver observer = new ConcreteObserver(subject);
        subject.Attach(observer);

        subject.State = 10; // خروجی: Subject's state changed to: 10
    }
}
```

3. builder Pattern

الگوی Builder : برای ساخت اشیاء پیچیده به‌صورت مرحله‌به‌مرحله استفاده می‌شود. این الگو زمانی مفید است که یک شیء نیاز به تنظیمات متعدد و اختیاری دارد و شما می‌خواهید فرآیند ساخت آن را از منطق اصلی جدا کنید تا کد خواناتر و انعطاف‌پذیرتر شود.

چرا از الگوی Builder استفاده می‌کنیم؟

جداسازی فرآیند ساخت: منطق ساخت شیء پیچیده را از کلاس اصلی جدا می‌کند.

فرض کنید می‌خواهیم یک شیء Pizza بسازیم که ویژگی‌های مختلفی مثل نوع خمیر، سس، و مواد روی پیتزا دارد. استفاده از سازنده‌های متعدد برای تنظیم همه ترکیبات ممکن پیچیده می‌شود، بنابراین از الگوی Builder استفاده می‌کنیم

```csharp
using System;
using System.Collections.Generic;

// کلاس نهایی که قرار است ساخته شود
public class Pizza
{
    public string Dough { get; private set; }
    public string Sauce { get; private set; }
    public List<string> Toppings { get; private set; }

    // سازنده خصوصی برای جلوگیری از ساخت مستقیم
    private Pizza() { Toppings = new List<string>(); }

    // نمایش اطلاعات پیتزا
    public void Display()
    {
        Console.WriteLine($"Pizza with {Dough} dough, {Sauce} sauce, and toppings: {string.Join(", ", Top
    }
```

```csharp
    // کلاس Builder
    public class PizzaBuilder
    {
        private readonly Pizza _pizza;

        public PizzaBuilder()
        {
            _pizza = new Pizza();
        }

        // متدهای زنجیره ای برای تنظیم ویژگی‌ها
        public PizzaBuilder SetDough(string dough)
        {
            _pizza.Dough = dough;
            return this; // برای زنجیره ای شدن
        }

        public PizzaBuilder SetSauce(string sauce)
        {
            _pizza.Sauce = sauce;
            return this;
        }
```

```csharp

        public PizzaBuilder AddTopping(string topping)
        {
            _pizza.Toppings.Add(topping);
            return this;
        }

        // متد نهایی برای ساخت شیء
        public Pizza Build()
        {
            if (string.IsNullOrEmpty(_pizza.Dough) || string.IsNullOrEmpty(_pizza.Sauce))
            {
                throw new InvalidOperationException("Dough and Sauce are required!");
            }
            return _pizza;
        }
    }
}
```

```csharp
class Program
{
    static void Main()
    {
        // استفاده از Builder برای ساخت پیتزا
        Pizza pizza = new Pizza.PizzaBuilder()
            .SetDough("Thin Crust")
            .SetSauce("Tomato")
            .AddTopping("Cheese")
            .AddTopping("Pepperoni")
            .AddTopping("Mushrooms")
            .Build();

        pizza.Display();
    }
}
```

کاربردهای عملی الگوی Builder

- **ساخت اشیاء پیچیده:**
- مثل تنظیمات یک درخواست (HttpClient مثل) HTTP که شامل هدرها، پارامترها، و بدنه است.

```csharp
var request = new HttpRequestBuilder()
    .SetUrl("https://api.example.com")
    .AddHeader("Authorization", "Bearer token")
    .SetMethod("POST")
    .SetBody("{ \"data\": \"value\" }")
    .Build();
```

CQRS

CQRS یا **Command Query Responsibility Segregation** یک الگوی طراحی نرم‌افزاری است که در معماری‌های نرم‌افزار، به‌ویژه در سیستم‌های پیچیده و مقیاس‌پذیر، استفاده می‌شود. این الگو مسئولیت‌های مربوط به دستورات (Commands)و پرس‌وجوها (Queries) را از هم جدا می‌کند. در ادامه به صورت خلاصه و جامع توضیح می‌دهم:

اجزای اصلی CQRS

Commands:

- دستوراتی هستند که حالت سیستم را تغییر می‌دهند (مثل ثبت کاربر جدید، به‌روزرسانی موجودی انبار).
- معمولاً به صورت یک درخواست ساده با داده‌های مورد نیاز ارسال می‌شوند.
- نتیجه‌ی یک Command معمولاً تغییر در پایگاه داده یا ارسال رویداد است، نه بازگشت داده.

Queries:

- درخواست‌هایی برای دریافت داده‌ها (مثل لیست کاربران یا جزئیات یک محصول).
- فقط داده را می‌خوانند و حالت سیستم را تغییر نمی‌دهند.
- معمولاً ساده و مستقیم هستند.

مزایای CQRS

مقیاس‌پذیری :

- می‌توانید پایگاه داده‌های خواندن و نوشتن را جدا کنید (مثلاً پایگاه داده NoSQL برای خواندن و SQL برای نوشتن)
- این امکان را فراهم می‌کند که هر بخش را به‌صورت جداگانه مقیاس‌بندی کنید.

Dependency Injection

تزریق وابستگی (Dependency Injection) یا (DI در سی‌شارپ یک الگوی طراحی است که به ما کمک می‌کند وابستگی‌های بین کلاس‌ها را مدیریت کنیم و کد را منعطف‌تر، قابل‌تست و قابل‌نگهداری کنیم.

انواع تزریق وابستگی

سه روش اصلی برای تزریق وابستگی در سی‌شارپ وجود دارد:

تزریق از طریق سازنده(Constructor Injection)

تزریق از طریق متد(Method Injection)

تزریق از طریق پراپرتی(Property Injection)

1. تزریق از طریق سازنده (Constructor Injection)

در این روش، وابستگی‌ها از طریق پارامترهای سازنده (Constructor) به کلاس تزریق می‌شوند. این رایج‌ترین نوع تزریق وابستگی است و برای زمانی مناسب است که وابستگی‌ها برای عملکرد کلاس ضروری هستند.

فرض کنید یک سرویس برای لاگ کردن پیام‌ها داریم و می‌خواهیم آن را به یک کلاس تزریق کنیم.

2. تزریق از طریق متد (Method Injection)

در این روش، وابستگی از طریق پارامترهای یک متد به کلاس تزریق می‌شود. این روش زمانی مفید است که وابستگی فقط برای یک متد خاص موردنیاز باشد.

3. تزریق از طریق پراپرتی (Property Injection)

در این روش، وابستگی از طریق یک پراپرتی عمومی (setter) به کلاس تزریق می‌شود. این روش زمانی استفاده می‌شود که وابستگی اختیاری باشد یا بخواهیم بعداً آن را تغییر دهیم.

IOC

کانتینر IoC(Inversion of Control) چارچوبی است که ایجاد و طول عمر اشیاء را در یک برنامه مدیریت می کند و از ایجاد و ردیابی اشیاء در یک برنامه مراقبت می کند. علاوه بر این، راهی برای register و resolve وابستگی ها در صورت نیاز فراهم می کند. در ASP.NET Core، کانتینر IoC در فریمورک تعبیه شده است و می توان آن را در فایل Program.cs پیکربندی کرد.

```
builder.Services.AddScoped<IUserService, UserService>();
```

در مثال بالا، ما در حال ثبت سرویسی به نام UserService هستیم که اینترفیس IUserService را پیاده سازی می کند. متد AddScoped به کانتینر می گوید که برای هر درخواست مشتری یک نمونه جدید از کلاس UserService ایجاد کند. روش‌های دیگری مانند AddSingleton و AddTransient برای ثبت خدمات با طول عمر متفاوت در دسترس هستند. در ادامه به سایر گزینه‌ها می‌پردازیم.

تزریق وابستگی از طریق سازنده یا Constructor Injection

Constructor Injection تکنیکی است که در آن وابستگی ها از طریق سازنده آن به یک کلاس ارائه می شود. این کار دیدن وابستگی های یک کلاس را آسان می کند و به کانتینر IoC اجازه می دهد تا وابستگی های مورد نیاز را به صورت خودکار ایجاد و تزریق کند.

```
public class UserController : ControllerBase
{
    private readonly IUserService _userService;
    public UserController(IUserService userService)
    {
        _userService = userService;
    }
{
```

در مثال بالا، کلاس UserController به اینترفیس IUserService وابستگی دارد. وابستگی از طریق سازنده تزریق می شود و کانتینر IoC به طور خودکار وابستگی مورد نیاز را ایجاد و تزریق می کند.

طول عمر وابستگی

در زمان ثبت سرویس، وابستگی ها نیاز به تعریف طول عمر دارند. طول عمر سرویس، شرایطی را که تحت آن یک نمونه جدید از سرویس ایجاد می شود را مشخص می کند. در زیر طول عمر تعریف شده توسط چارچوب . NET Core DI آمده است.

Singleton

متد AddSingleton یک سرویس را از طریق کانتینر تزریق وابستگی به صورت Singleton ثبت می کند. Singleton یک شی است که فقط یک بار ایجاد می شود و همان نمونه در کل برنامه به اشتراک گذاشته می شود. این به این معنی است که هر مؤلفه ای که به سرویس سینگلتون ثبت شده وابسته باشد، همان نمونه سرویس را دریافت خواهد کرد.

```
builder.Services.AddSingleton<IEmailService, EmailService>();
```

توجه به این نکته مهم است که سرویس singleton هنگام شروع برنامه ایجاد می شود و تا زمان خاموش شدن از بین نمی رود. این به این معنی است که هر منبعی که توسط سرویس singleton استفاده می‌شود در طول عمر برنامه در حافظه نگهداری می‌شود. بنابراین، هنگام استفاده از singleton ها باید مراقب باشیم تا از memory leak یا مشکلات عملکردی جلوگیری کنیم.

Scoped

یک سرویس scoped یک بار در هر درخواست کلاینت ایجاد می شود و در آن درخواست به اشتراک گذاشته می شود.

```
builder.Services.AddScoped<IUserService, UserService>();
```

سرویس های Scoped هنگام به اشتراک گذاری داده ها یا منابع در یک درخواست کلاینت مفید هستند. به عنوان مثال، ما یک کانتکست پایگاه داده داریم که می خواهیم برای یک درخواست کلاینت استفاده کنیم. با ثبت کانتکست پایگاه داده به عنوان یک سرویس scoped ، می‌توانیم اطمینان حاصل کنیم که همه مؤلفه‌هایی که از کانتکست در یک درخواست کلاینت استفاده می‌کنند، از نمونه مشابهی از کانتکست استفاده می‌کنند.

```
builder.Services.AddDbContext<DataContext>(opt=>
opt.UseSqlServer(builder.Configuration.GetConnectionString("DataContext")));
```

Transient

یک سرویس Transient هر زمان که درخواست شود ایجاد می شود و بین اجزا یا درخواست های مختلف به اشتراک گذاشته نمی شود.

```
builder.Services.AddTransient<IPasswordGeneratorService, PasswordGeneratorService>();
```

سرویس‌های Transient زمانی که می‌خواهیم در هر لحظه یک نمونه جدید از یک سرویس را ایجاد کنیم مفید هستند. در مثال بالا، ما سرویسی داریم که رمزهای عبور تصادفی تولید می کند. با ثبت PasswordGeneratorService به عنوان یک سرویس Transient ، می توانیم اطمینان حاصل کنیم که هر بار که سرویس درخواست می شود، یک نمونه جدید از آن ایجاد می شود.

RabbitMQ

یک نرم‌افزار واسطه است که امکان تبادل پیام‌ها بین سیستم‌ها، برنامه‌ها یا سرویس‌های مختلف را فراهم می‌کند. این سیستم به ما کمک می کنه که بین چندین برنامه مختلف که حتی با زبان های مختلفی هم نوشته شدن بتونیم ارتباط برقرار کنیم البته توی RabbitMQ چیزی وجود داره به نام صف یا همون queue که پیام هایی که ما برای برنامه های دیگه ارسال می کنیم درون یک صف قرار می ده و برنامه مقصد به ترتیب و طبق تنظیماتی که ما براش تعریف می کنیم اون پیام هارو میخونه و پردازش میکنه (البته اگه نیاز به پردازش داشته باشه).

RabbitMQ یکی از بهترین انتخاب ها برای ارتباط برقرار کردن بین microservice ها می باشد. و هم اکنون خیلی ها ازش استفاده میکنن.

ما دوتا مفهوم اصلی توی این سیستم داریم

1-producers

2-consumers

producers کسی یا برنامه ای هست که پیام اولیه رو تولید میکنه و به صف پردازش ارسال میکنه و consumers کسی یا برنامه ای هست که این پیام هارو به ترتیب از صف میگیره و پردازش میکنه.

بین producers و consumers ۲ تا مفهوم وجود داره به نام های exchange و queue که قبلا درباره queue توضیح دادم که یک صف که درخواست توی اون جمع میشه و consumers ها به ترتیب اونارو بر میدارن و پردازش می کنند. ولی exchange یکم توضیح سخته نیاز هست که طی فرآیند آموزش باهاش درگیر بشید تا یاد بگیرید اما اینو بدونید که اصلا چیز پیچیده ای نیست به طور کلی روش ارسال پیام از producers به صف هاست که ۴ تا حالت داره Direct - Fanout - Topic - Headers که هرکدوم مزایا و معایب خودشونو دارن.

Exchange

در RabbitMQ، Exchange یکی از اجزای اصلی است که نقش واسطه‌ای بین تولیدکننده (Producer) و صف‌ها (Queues) را ایفا می‌کند. Exchange پیام‌هایی که از تولیدکننده دریافت می‌کند را بر اساس قوانین خاصی (Routing Rules) به صف‌های مناسب هدایت می‌کند. به عبارت دیگر، Exchange تعیین می‌کند که یک پیام به کدام صف یا صف‌ها ارسال شود.

عملکرد Exchange

دریافت پیام: Producer پیام را به همراه یک Routing Key به Exchange ارسال می‌کند.

تصمیم‌گیری برای هدایت: Exchange بر اساس نوع خود (Direct، Topic و غیره) و قوانین Binding تصمیم می‌گیرد که پیام به کدام صف یا صف‌ها ارسال شود.

ارسال به Queue: پیام به صف‌های مناسب منتقل می‌شود تا توسط Consumer پردازش شود.

در صورت عدم تطبیق: اگر هیچ صفی برای پیام پیدا نشود، بسته به تنظیمات، پیام ممکن است حذف شود یا به یک صف پیش‌فرض (Dead Letter Queue) هدایت شود.

1.Direct Exchange

نحوه کار :پیام را به صفی هدایت می‌کند که Routing Key آن دقیقاً با Binding Key مطابقت داشته باشد .

ویژگی‌ها :

- تطبیق یک‌به‌یک بین Routing Key پیام و Binding Key صف.
- اگر چندین صف با همان Routing Key به Exchange متصل باشند، پیام به همه آن‌ها ارسال می‌شود.

کاربرد :

- وقتی نیاز دارید پیام‌ها به‌طور دقیق به یک یا چند صف خاص بروند.
- مثال: هدایت لاگ‌های "error" فقط به صف."error_queue"

2. Topic Exchange

این Exchange پیام‌ها را به یک یا چند صف می‌فرستد. نحوه تصمیم گیری برای اینکه پیام را به کدام صف ارسال کند بستگی به کلید مسیریابی و همچنین الگویی است که صف را به Exchange متصل می کند. این نوع Exchange برای پیاده‌سازی الگوهای publish/subscribe مناسب است. همچنین این نوع Exchange برای مسیریابی های multicast پیام‌ها مناسب است. حالتی را تصور کنید که برای یک پیام چندین مصرف کننده وجود دارد که می‌توان انتخاب کرد که پیام را به کدام داد، برای این کار از این الگو استفاده می شود.

3. Fanout Exchange

یک Fanout Exchange یک کپی از پیام دریافت شده را بدون درنظر گرفتن Routing Key برای همه Queue های Bindشده به این Exchange ارسال میکند.

این Exchangeزمانی مورد استفاده است که می خواهیم Message های مشابه را به یک یا چند Queue با Consumerهایی که هر کدام پیام های مشابه را به روش های مختلف پردازش می کنند ارسال کنیم.

4. Header Exchange

یک Header Exchange براساس آرگومان های Header هر Request و مقادیر اختیاری، پیام ها را Route میکند. مشابه Topic Exchenge ها، Header Exchenge موقع تصمیم گیری برای Route کردن پیام ها به جای درنظر گرفتن Routing key، مقادیر Header را درنظر میگیرد. برای انطباق بررسی میکند که مقادیر Header با مقادیر مشخص شده در Binding برابر باشند.

Repository و Unit of Work

در توسعه نرم‌افزار، به‌ویژه در پروژه‌های مبتنی بر معماری‌های شیءگرا و سیستم‌های پایگاه داده، الگوهای Repository و Unit of Work برای مدیریت دسترسی به داده‌ها و تراکنش‌ها استفاده می‌شوند. این دو الگو معمولاً در کنار هم به کار می‌روند تا کد را تمیزتر، قابل نگهداری‌تر و تست‌پذیرتر کنند.

1. Repository Pattern

هدف استفاده از الگوی repository pattern در C# این است که این وابستگی را از بین برده و امکان تفکیک کامل منطق دسترسی به داده‌ها (Data Access Logic) از لایه کسب‌وکار را فراهم می‌کند. این الگو یک لایه انتزاعی (Abstraction Layer)بین بخش‌های کسب‌وکار و پایگاه داده ایجاد می‌کند.

2.Unit Of Work

Unit of Work مثل یک هماهنگ‌کننده عمل می‌کنه که تمام عملیات روی داده‌ها (مثل درج، به‌روزرسانی، حذف) رو در یک تراکنش واحد جمع‌آوری می‌کنه و تضمین می‌کنه که همه تغییرات یا با هم اعمال بشن یا اصلاً اعمال نشن (Atomicity).

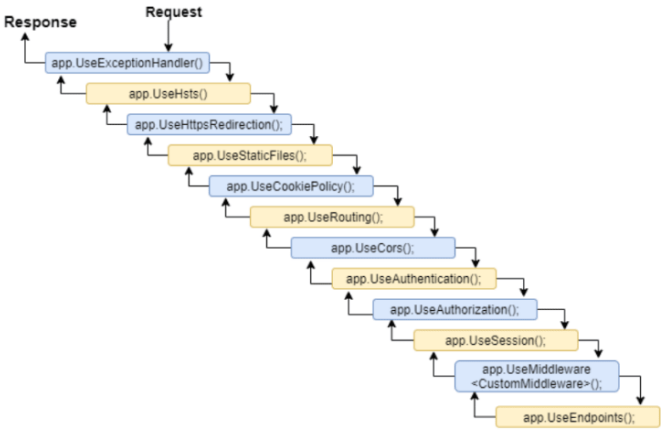
کاربرد: به جای اینکه هر Repository به‌صورت جداگانه با دیتابیس کار کنه و تغییرات رو ذخیره کنه، Unit of Work همه این عملیات رو مدیریت می‌کنه و در نهایت با یک دستور (مثل SaveChanges) همه رو به دیتابیس اعمال می‌کنه.

چرا از Unit of Work استفاده می‌کنیم؟ برای کاهش پیچیدگی، مدیریت تراکنش‌ها، و جداسازی لایه داده از منطق برنامه.

رابطه با Entity Framework: در EF، خود DbContext تا حدی نقش Unit of Work رو بازی می‌کنه، چون تغییرات رو ردیابی و با SaveChanges اعمال می‌کنه. اما تعریف صریح Unit of Work برای پروژه‌های بزرگ و پیچیده خوانایی و انعطاف‌پذیری رو بیشتر می‌کنه.

Middleware

MiddleWare یکی از مفاهیم اساسی در معماری ASP.NET Core است که به شما اجازه می‌دهد درخواست‌ها (Request)و پاسخ‌ها (Response) را قبل از رسیدن به مقصد پردازش کنید .Middleware در حقیقت اجزایی است که در زنجیره پردازش درخواست و پاسخ قرار می‌گیرد و وظایف خاصی مانند احراز هویت، لاگ‌گیری، کش کردن، و حتی مدیریت خطاها را انجام می‌دهد.



Short-Circuiting در Middleware در ASP.NET Core

Short-Circuiting در Middleware به حالتی گفته می‌شه که یک Middleware در Request Pipeline پردازش درخواست رو متوقف می‌کنه و مستقیماً یک پاسخ (Response) تولید می‌کنه، بدون اینکه درخواست به Middlewareهای بعدی یا بقیه Pipeline منتقل بشه. این مکانیزم برای بهینه‌سازی یا مدیریت شرایط خاص (مثل خطاها یا احراز هویت) استفاده می‌شه.

چطور کار می‌کنه؟

- درASP.NET Core ، هر Middleware می‌تونه تصمیم بگیره که درخواست رو به Middleware بعدی پاس بده (با فراخوانی ()nextیا ((next.Invokeیا خودش پاسخ رو تولید کنه و Pipeline رو "کوتاه کنه."
- وقتی یک Middleware پاسخ رو تولید می‌کنه (مثلاً با تنظیم HttpContext.Responseو ()nextرو فراخوانی نمی‌کنه، Pipelineمتوقف می‌شه و پاسخ به کلاینت برگردونده می‌شه.

LINQ

LINQ مخفف Language Integrated Query یک قابلیت قدرتمند در #C است که امکان نوشتن پرس‌وجوهای (Queries)خوانا و مختصر برای کار با داده‌ها رو فراهم می‌کنه .بهت اجازه می‌ده با داده‌ها مثل لیست‌ها، آرایه‌ها، دیتابیس‌ها، XML، یا حتی (JSON به‌صورت یکپارچه و با سینتکس مشابه SQL کار کنی، بدون اینکه نیاز به نوشتن کدهای پیچیده داشته باشی.

تفاوت Deferred Execution و Immediate Execution در LINQ

در LINQ، نحوه اجرای پرس‌وجوها به دو دسته **Deferred Execution**(اجرای به‌تعویق‌افتاده) و **Immediate Execution** (اجرای فوری) تقسیم می‌شه. این تفاوت به زمان اجرای پرس‌وجو و نحوه پردازش داده‌ها بستگی داره.

Deferred Execution

در این حالت، پرس‌وجوی LINQ فقط تعریف می‌شه و تا زمانی که داده‌ها واقعاً مورد نیاز نباشن (مثلاً با پیمایش یا تبدیل به مجموعه)، اجرا نمی‌شه. این رفتار به نام Lazy Evaluation هم شناخته می‌شه.

ویژگی‌ها :

- پرس‌وجو تا زمانی که متدهایی مثل ToList، ToArray، یا حلقه foreachفراخوانی نشن، اجرا نمی‌شه.
- بهینه برای عملکرد، چون فقط وقتی نیازه پردازش انجام می‌شه.
- امکان تغییر پرس‌وجو قبل از اجرا وجود داره.
- مناسب برای کار با داده‌های بزرگ یا دیتابیس، چون از پردازش غیرضروری جلوگیری می‌کنه.

Immediate Execution

در این حالت، پرس‌وجوی LINQ فوراً اجرا می‌شه و نتیجه در حافظه ذخیره می‌شه.

ویژگی‌ها :

- نتیجه پرس‌وجو بلافاصله محاسبه و ذخیره می‌شه (مثلاً به‌صورت لیست یا مقدار).
- برای زمانی مناسبه که می‌خوای نتیجه رو فوراً داشته باشی یا داده‌ها ثابت بشن.
- مصرف حافظه بیشتری داره، چون داده‌ها فوراً پردازش و ذخیره می‌شن.

متدهای مرتبط :متدهایی مثل ToList، ToArray، Count، Sum، First، FirstOrDefault، Max، Min، Average.

تفاوت‌های کلیدی		
ویژگی	Deferred Execution	Immediate Execution
زمان اجرا	وقتی داده‌ها مورد نیاز باشن (مثل `foreach`)	فوراً در زمان تعریف پرس‌وجو
متدها	`Where`, `Select`, `OrderBy`, `GroupBy`	`ToList`, `Count`, `First`, `Sum`
تأثیر تغییرات داده	تغییرات داده قبل از اجرا اثر می‌ذاره	تغییرات داده بعد از اجرا اثر نداره
مصرف حافظه	بهینه‌تر، چون فقط در صورت نیاز اجرا می‌شه	مصرف حافظه بیشتر، چون نتیجه ذخیره می‌شه
کاربرد	مناسب برای داده‌های پویا یا بزرگ	مناسب برای وقتی نیاز به نتیجه ثابت داری

چرا از Any به‌جای Count > 0 استفاده می کنیم؟

Any بررسی می‌کنه که آیا حداقل یک عنصر با شرط مطابقت داره و بعد از پیدا کردن اولین مورد متوقف می‌شه. اما Count کل مجموعه رو می‌شمره که می‌تونه زمان‌بر باشه، به‌خصوص برای داده‌های بزرگ. Any بهینه‌تره.

تفاوت بین First و FirstOrDefault چیه؟ کی از کدوم استفاده کنیم؟

First اولین عنصر منطبق با شرط رو برمی‌گردونه و اگه هیچ عنصری پیدا نشه، خطای InvalidOperationException می‌ده. FirstOrDefault هم اولین عنصر رو برمی‌گردونه، ولی اگه چیزی پیدا نشه، مقدار پیش‌فرض (مثل null برای نوع مرجع) برمی‌گردونه. از FirstOrDefault وقتی مطمئن نیستی داده‌ای وجود داره استفاده کن تا از خطا جلوگیری بشه.

تفاوت بین IEnumerable و IQueryable در LINQ چیه؟

برای کار با داده‌های در حافظه (مثل لیست‌ها) استفاده می‌شه و پرس‌وجوها روی کلاینت اجرا می‌شن. IQueryable برای کار با منابع خارجی (مثل دیتابیس) استفاده می‌شه و پرس‌وجوها به SQL ترجمه شده و روی سرور اجرا می‌شن. IQueryable بهینه‌تره چون فقط داده‌های موردنیاز رو از دیتابیس می‌گیره.

تفاوت SingleOrDefault و FirstOrDefault در LINQ

FirstOrDefault: اولین عنصر از مجموعه که با شرط داده‌شده مطابقت داره رو برمی‌گردونه. اگه هیچ عنصری پیدا نشه، مقدار پیش‌فرض (مثل null برای انواع مرجع) برمی‌گردونه.

SingleOrDefault: دقیقاً یک عنصر از مجموعه که با شرط داده‌شده مطابقت داره رو برمی‌گردونه. اگه هیچ عنصری پیدا نشه، مقدار پیش‌فرض (مثل null) برمی‌گردونه. اگه بیش از یک عنصر پیدا بشه، خطای InvalidOperationException می‌ده. کل مجموعه رو بررسی می‌کنه تا مطمئن بشه فقط یک عنصر با شرط مطابقت داره.

Entity Framework

قبل از Entity Framework بهتره با مفهوم ORM یا Object Relational Mapping اشنا بشیم. ORM ابزاریه که به وسیله اون میشه جداول پایگاه داده رو پردازش کرد و اونها رو به فرمت شی نگاشت که در کدنویسی خواناتر باشند. در کدنویسی به جای این که به صورت مستقیم با جداول پایگاه داده کار کنیم، بر روی اشیا کار میکنیم. به این اشیا Entity گفته میشه.

Entity Framework چیست؟

EF یک ORM است که امکان کار با دیتابیس رو با اشیاء #C فراهم می‌کنه. ازش استفاده می‌کنیم چون کد رو ساده‌تر می‌کنه، کوئری‌های نوع‌امن با LINQ ارائه می‌ده، و مدیریت روابط و تغییرات رو خودکار می‌کنه.

نکته مصاحبه: روی مزایای خوانایی و صرفه‌جویی در زمان تأکید کن.

اجزای اصلی EF

DbContext: قلب EF، که ارتباط با دیتابیس رو مدیریت می‌کنه و تغییرات رو ردیابی می‌کنه

DbSet: نماینده یک جدول در دیتابیس، که برای کوئری و عملیات CRUD استفاده می‌شه.

Navigation Properties: برای تعریف روابط بین جداول (مثل One-to-Many یا Many-to-Many).

تفاوت Lazy Loading و Eager Loading در EF چیه؟ کی از کدوم استفاده کنیم؟

Lazy Loading برای سناریوهایی که نمی‌دونی چه داده‌ای نیاز داری مناسبه. Eager Loading برای وقتی که از قبل می‌دونی داده‌های مرتبط لازمن و می‌خوای تعداد کوئری‌ها رو کاهش بدی.

مشکل N+1 در Lazy Loading در Entity Framework

مشکل N+1 یکی از مسائل رایج در کار با Entity Framework (EF) و Lazy Loading (بارگذاری تنبل) است که می‌تونه عملکرد برنامه رو به شدت تحت تأثیر قرار بده. این مشکل وقتی رخ می‌ده که برای دسترسی به داده‌های مرتبط (مثل Navigation Properties)، به ازای هر رکورد اصلی، کوئری‌های اضافی به دیتابیس ارسال بشه. در ادامه این مشکل رو با جزئیات توضیح می‌دم

مثال ساده

فرض کن دو جدول داری: `Users` و `Orders` با رابطه یک‌به‌چند. هر کاربر می‌تونه چند سفارش داشته باشه.

csharp — Collapse | Wrap | Copy

```csharp
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public virtual List<Order> Orders { get; set; } // Lazy Loading با virtual
}

public class Order
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public string Product { get; set; }
}
```

حالا فرض کن می‌خوای تمام کاربران و محصولات سفارش‌هاشون رو نمایش بدی:

csharp — Collapse | Wrap | Copy

```csharp
using (var context = new AppDbContext())
{
    var users = context.Users.ToList(); // 1 کوئری برای لود کاربران
    foreach (var user in users)
    {
        Console.WriteLine($"User: {user.Name}");
        foreach (var order in user.Orders) // N کوئری برای لود Orders هر کاربر
        {
            Console.WriteLine($"Order: {order.Product}");
        }
    }
}
```

مشکل :

- کوئری 1: لود تمام کاربران (مثلاً 100 کاربر).
- کوئری‌های N: به ازای هر کاربر، یک کوئری برای لود Orders (100 کوئری اضافی.)
- **جمعاً** 101 = 100 + 1 :کوئری به دیتابیس.

راه‌حل‌های رفع مشکل N+1

Include با Eager Loading

داده‌های مرتبط رو در همون کوئری اولیه با استفاده از Include یا ThenInclude لود کن.

```csharp
var users = context.Users
    .Include(u => u.Orders) // لود Orders در یک کوئری
    .ToList();
foreach (var user in users)
{
    Console.WriteLine($"User: {user.Name}");
    foreach (var order in user.Orders) // بدون کوئری اضافی
    {
        Console.WriteLine($"Order: {order.Product}");
    }
}
```

تفاوت بین ردیابی تغییرات (Change Tracking) و حالت No-Tracking چیه؟ چطور می‌تونی رفتار ردیابی رو شخصی‌سازی کنی؟

Change Tracking: EF به‌طور پیش‌فرض تغییرات اشیاء (مثل اضافه، حذف، به‌روزرسانی) رو ردیابی می‌کنه و موقع SaveChanges به دیتابیس اعمال می‌کنه. این کار حافظه بیشتری مصرف می‌کنه، چون EF باید وضعیت اشیاء رو در حافظه نگه داره.

No-Tracking: با AsNoTracking، EF اشیاء رو ردیابی نمی‌کنه، که باعث کاهش مصرف حافظه و بهبود عملکرد برای کوئری‌های فقط خواندنی می‌شه

C#

partial و sealed و abstract

کلاس abstract

- **تعریف** :کلاسی که با کلمه کلیدی abstract تعریف می‌شود، نمی‌تواند نمونه‌سازی (instantiate) شود و برای استفاده به‌عنوان پایه برای ارث‌بری طراحی شده است.

ویژگی‌ها :

- می‌تواند شامل متدهای پیاده‌سازی‌شده یا متدهای abstract(بدون پیاده‌سازی) باشد.

کلاس sealed

- **تعریف** :کلاسی که با کلمه کلیدی sealed تعریف می‌شود، نمی‌تواند به‌عنوان کلاس پایه برای ارث‌بری استفاده شود.
- **ویژگی‌ها** :
  - از ارث‌بری جلوگیری می‌کند، یعنی هیچ کلاسی نمی‌تواند از آن ارث ببرد.

ویژگی های abstract ---- sealed

"نمونه‌سازی","نمی‌توان نمونه ساخت"----"می‌توان نمونه ساخت"

"ارث‌بری","برای ارث‌بری طراحی شده"----"از ارث‌بری جلوگیری می‌کند"

"متدها","می‌تواند متدهای abstract داشته باشد"----"نمی‌تواند متد abstract داشته باشد"

"کاربرد","برای تعریف کلاس‌های پایه عمومی"----"برای محدود کردن تغییرات و ارث‌بری"

"انعطاف‌پذیری","انعطاف‌پذیر برای گسترش"----"غیرقابل گسترش"

سناریوی استفاده

- از abstract وقتی استفاده کن که بخوای یک الگوی پایه برای گروهی از کلاس‌ها تعریف کنی (مثل کلاس Animal برای Dog و Cat).
- از sealed وقتی استفاده کن که بخوای کلاس رو قفل کنی و نذاری کسی رفتارش رو تغییر بده(مثل یک کلاس ابزار یا Utility که نباید تغییر کنه.)

Partial

کلاس Partial در #C امکانی فراهم می‌کنه که تعریف یک کلاس، یا متد رو بتونی در چندین فایل مختلف تقسیم کنی. این قابلیت بهت اجازه می‌ده کد رو سازمان‌دهی کنی و خوانایی و نگهداری اون رو بهبود بدی، به‌خصوص در پروژه‌های بزرگ یا وقتی چند توسعه‌دهنده روی یک پروژه کار می‌کنن.

وقتی یک کلاس خیلی بزرگ می‌شه، می‌تونی منطقش رو بین چند فایل تقسیم کنی تا خوانایی و مدیریت کد بهتر بشه. مثال: یه فایل برای متدهای اصلی، یه فایل برای متدهای کمکی.

در زمان کامپایل، کامپایلر تمام بخش‌های Partial یک کلاس رو ترکیب می‌کنه و به‌عنوان یک کلاس واحد می‌بینه

همه بخش‌ها باید دسترسی یکسانی داشته باشن (مثلاً همه public یا همه private).

سوالات احتمالی مصاحبه

- **کلاس Partial چه زمانی استفاده می‌شه؟** پاسخ: برای تقسیم کد، کار با ابزارهای تولید کد، یا پروژه‌های تیمی.
- **تفاوت Partial Class با ارث‌بری چیه؟**پاسخ Partial :برای تقسیم یک کلاس به چند فایل استفاده می‌شه، اما ارث‌بری برای تعریف سلسله‌مراتب کلاس‌هاست.
- **اگه دو بخش Partial سطح دسترسی متفاوتی داشته باشن، چی می‌شه؟**پاسخ: خطای کامپایل می‌گیرید.

تفاوت بین string و StringBuilder در #C چیست؟ در چه سناریوهایی از StringBuilder استفاده می‌کنیم؟

string غیرقابل تغییر (immutable) است و هر تغییر روی آن یک شیء جدید ایجاد می‌کند، که برای عملیات زیاد روی رشته‌ها ناکارآمد است. StringBuilder قابل تغییر (mutable) است و برای عملیات مکرر مثل الحاق رشته‌ها مناسب‌تره، چون حافظه کمتری مصرف می‌کنه.

سطوح دسترسی در#C

1- Public

عضو یا کلاس با سطح دسترسی public از همه‌جا قابل دسترسیه، چه داخل پروژه باشه، چه در اسمبلی‌های دیگه. کاربرد: وقتی می‌خوای یه کلاس یا عضو برای همه (حتی کدهای خارجی) قابل استفاده باشه، مثلاً درAPI های عمومی یا کتابخانه‌ها.

2- Protected

تعریف: عضو با سطح دسترسی protected فقط در کلاس خودش و کلاس‌های مشتق‌شده (فرزند) قابل دسترسیه، چه در همان اسمبلی باشن، چه در اسمبلی‌های دیگه.

کاربرد :برای ارث‌بری، وقتی می‌خوای یه عضو فقط برای کلاس‌های فرزند قابل دسترسی باشه، نه برای کدهای خارجی.

3- Internal

تعریف: عضو یا کلاس با سطح دسترسی internal فقط در همان اسمبلی (پروژه) قابل دسترسیه. خارج از اسمبلی نمی‌تونن بهش دسترسی داشته باشن.

کاربرد: وقتی می‌خوای یه عضو یا کلاس فقط داخل پروژه خودت استفاده بشه، مثلاً در کتابخانه‌های داخلی یا ماژول‌های یک پروژه.

نکات:

- برای محدود کردن دسترسی به خارج از اسمبلی مناسبه.
- در پروژه‌های بزرگ، برای جداسازی ماژول‌ها استفاده می‌شه.

4- Protected internal

تعریف: عضو در همان اسمبلی از همه‌جا قابل دسترسیه، و در اسمبلی‌های دیگر فقط از کلاس‌های مشتق‌شده قابل دسترسیه.

5- Private protected

تعریف: عضو فقط در کلاس‌های مشتق‌شده داخل همان اسمبلی قابل دسترسیه.

کاربرد: محدودترین سطح دسترسی، برای وقتی که می‌خوای فقط کلاس‌های فرزند در همان پروژه به عضو دسترسی داشته باشن.

جدول مقایسه

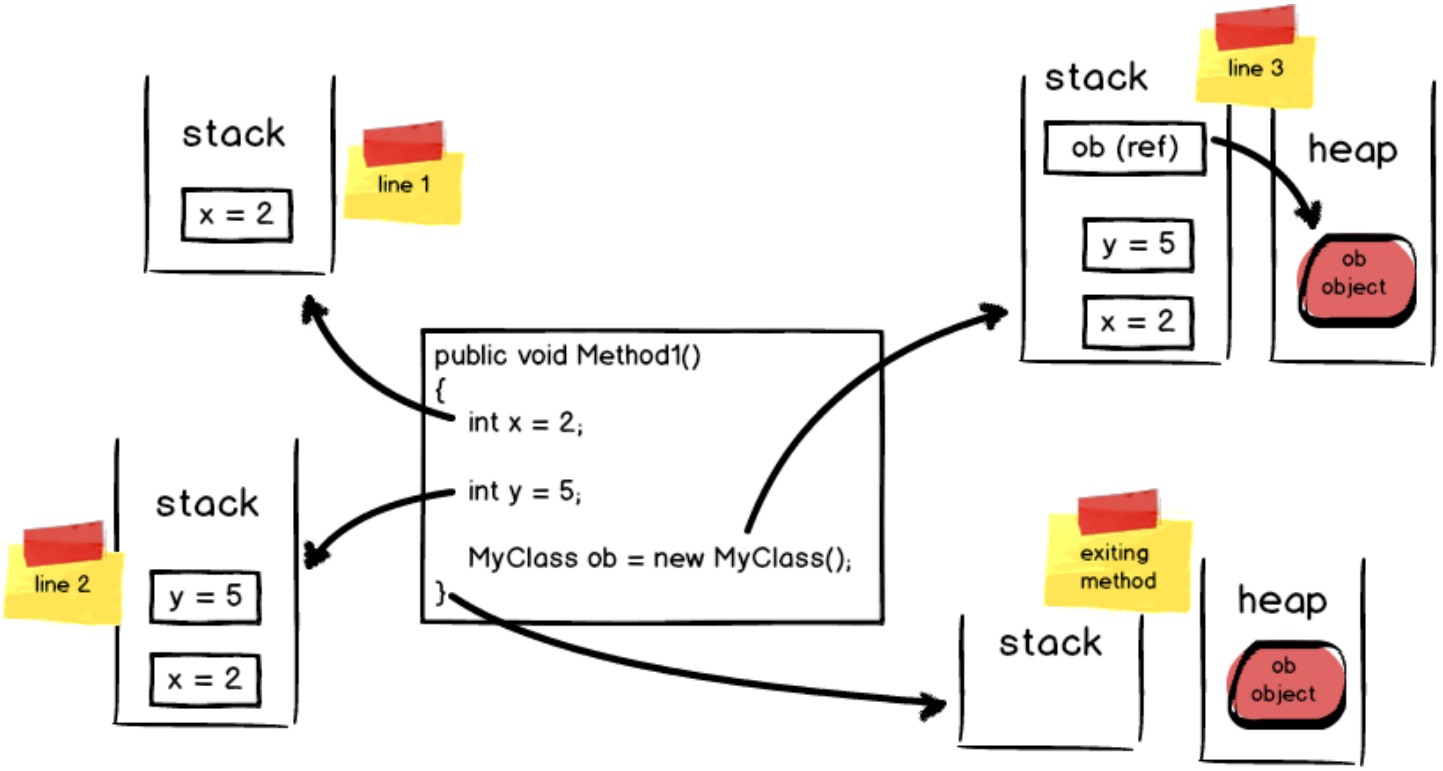
سطح دسترسی	دسترسی در همان کلاس	دسترسی در کلاس‌های مشتق‌شده	دسترسی در همان اسمبلی	دسترسی در اسمبلی‌های دیگر
public	✅	✅	✅	✅
protected	✅	✅	✅	✅ (فقط مشتق‌شده)
internal	✅	✅	✅	❌
protected internal	✅	✅	✅	✅ (فقط مشتق‌شده)
private protected	✅	✅ (فقط در همان اسمبلی)	✅ (فقط مشتق‌شده)	❌
private (پیش‌فرض)	✅	❌	❌	❌

سوال احتمالی مصاحبه

"اگه بخوای یه متد فقط برای کلاس‌های فرزند در همان پروژه قابل دسترسی باشه، از چه سطح دسترسی استفاده می‌کنی؟" (پاسخ: private protected)

حافظه Stack و Heap

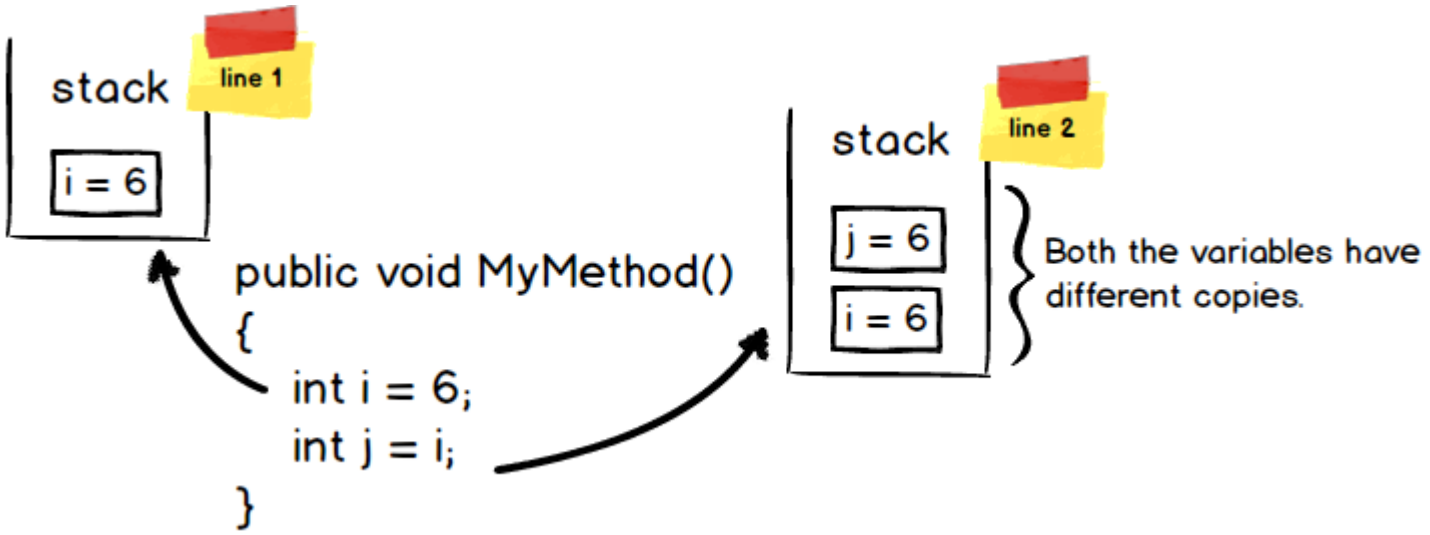
دو نوع تخصیص حافظه وجود دارد که یکی stack memory و دیگری heap memory است.



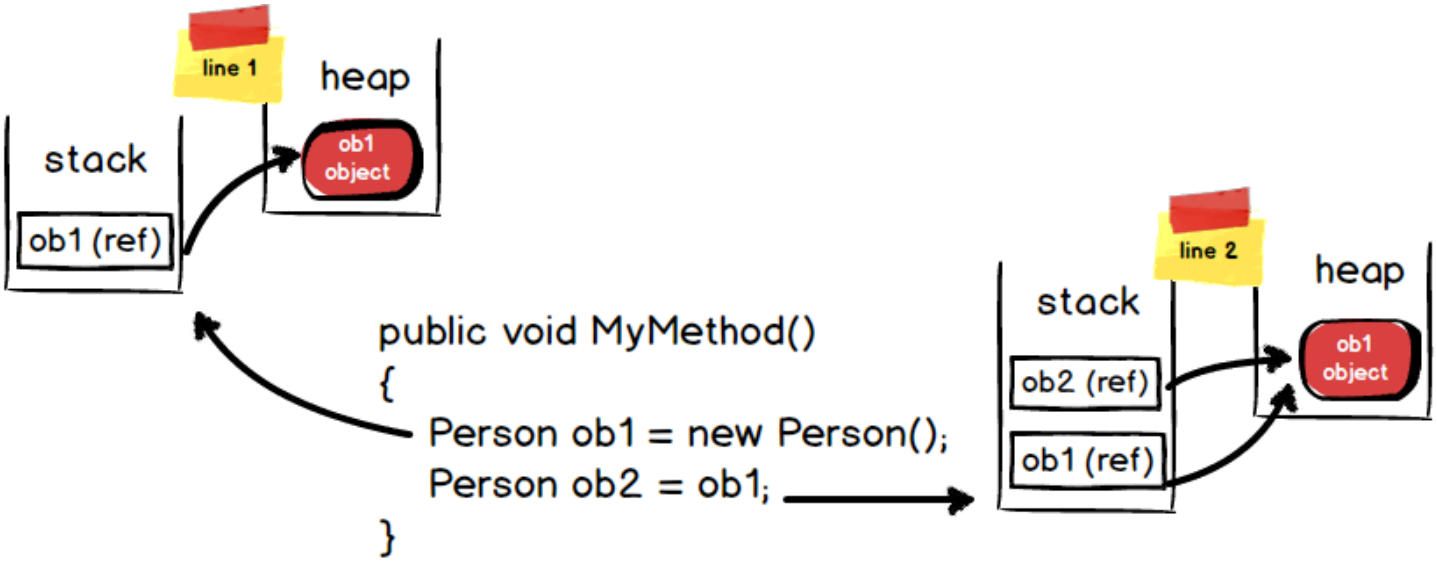
ممکن است برای‌تان سوال باشد که چرا stack و heap ؟ نمی‌شود همه در یک‌جا ذخیره شوند؟ اگر با دقت نگاه کنید می‌بینید که data type های اصلی (value types)، پیچیده و سنگین نیستند. آن‌ها مقادیر تکی مثل int i = 5 را نگه می‌دارند در حالی‌که object data types یا reference types پیچیده‌تر و سنگین‌تر هستند، آن‌ها به اشیای دیگری رجوع می‌کنند. به عبارت دیگر، آن‌ها به چندین مقدار رجوع می‌کنند. (زیرا اشیاء می‌توانند شامل مقادیر زیادی از فیلد و متد و... باشند) که هرکدام از آن‌ها باید در حافظه ذخیره شده باشد.

Value types و Reference types

Value type ها تمام و کمال در stack ذخیره می‌شوند، یعنی هم مقدار و هم متغیر همه‌گی یک‌جا هستند اما در reference type متغیر در stack است درحالی‌که object در heap قرار می‌گیرد و متغیر و شیء به هم متصل می‌شوند (متغیر به شیء اشاره می‌کند).

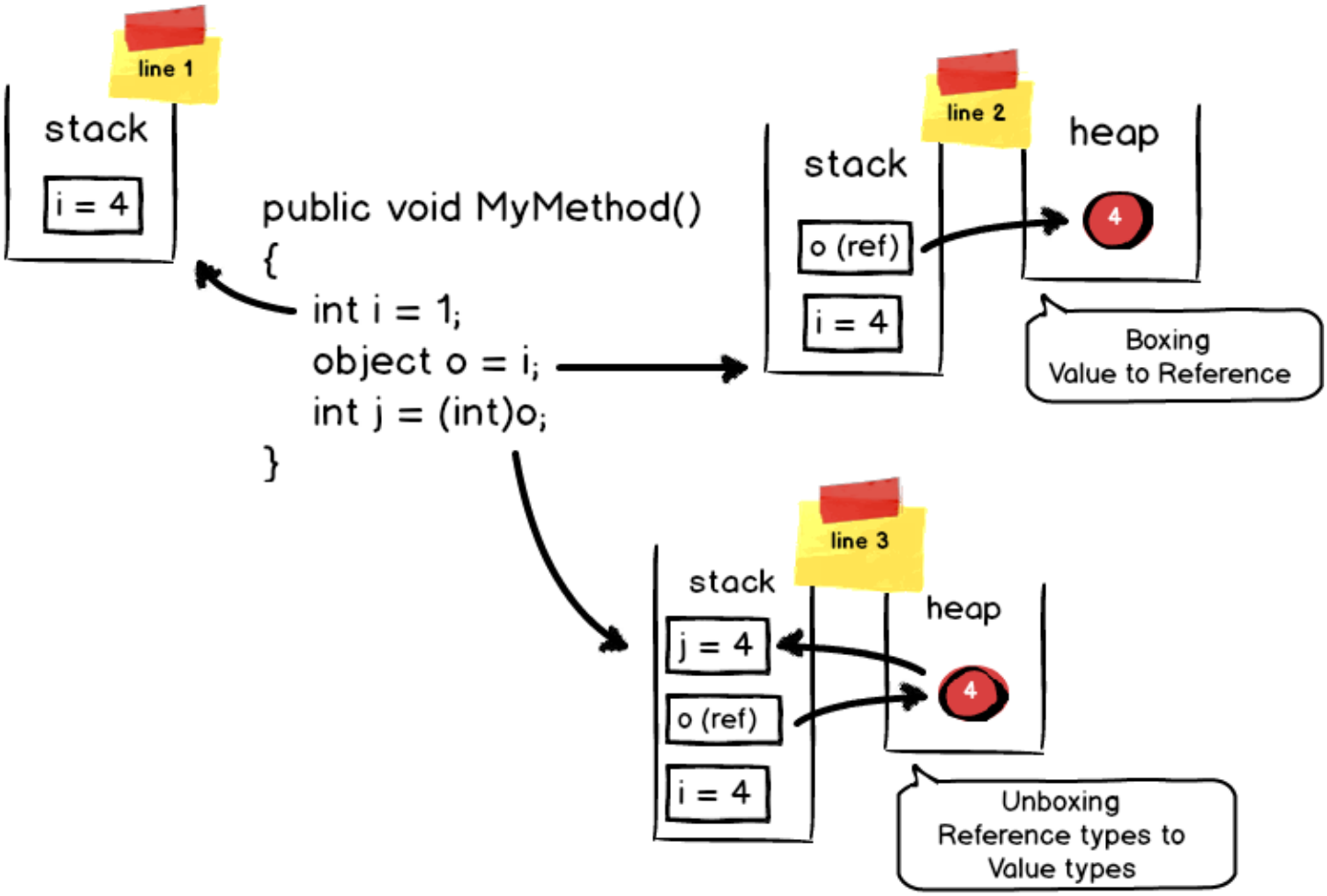


هنگامی‌که یک شیء می‌سازید و reference آن را با یک reference دیگر مساوی قرار می‌دهید، آن‌گاه هر دوی این reference ها به یک شیء رجوع می‌کنند و تغییر هر کدام از آن‌ها باعث تغییر شیء می‌شود زیرا هردو reference به یک شیء اشاره می‌کنند.



Boxing and Unboxing

به‌طور خلاصه، وقتی‌که یک مقدار value type را تبدیل به reference type می‌کنید، در واقع اطلاعات را از stack به heap می‌برید و هنگامی‌که یک مقدار reference type را تبدیل به value type می‌کنید، اطلاعات را از heap به stack می‌برید. این رفت و برگشت اطلاعات از stack به heap روی performance (کارایی، سرعت اجرا) برنامه تاثیر می‌گذارد. فرستادن اطلاعات از stack به heap در اصطلاح boxing و فرستادن اطلاعات از heap به stack در اصطلاح unboxing نامیده می‌شود.



Garbage Collection

نوعی مدیریت حافظه‌ی خودکار محسوب می‌شود. هربار که یک شیء می‌سازید، object شما در heap ذخیره می‌شود. تا زمانی‌که فضای کافی برای ذخیره‌ی این اشیاء داشته باشید می‌توانید شیء جدید بسازید اما همان‌طور که می‌دانید حافظه نامحدود نیست و ممکن است پر شود.

تفاوت‌های اساسی بین ref و out چیست؟

نیاز به مقداردهی اولیه

یکی از تفاوت‌های کلیدی بین این دو، الزام به مقداردهی اولیه است:

پارامترهای ref پیش از ارسال به متد باید مقداردهی شوند. اگر متغیر بدون مقداردهی باشد، کامپایلر با خطا مواجه خواهد شد. در مقابل، پارامترهای out نیازی به مقدار اولیه ندارند. این پارامترها در داخل متد مقداردهی شده و سپس مورد استفاده قرار می‌گیرند.